

Lesson 5 Assessment

Build & Validate a Production Database Integration Workflow

Database: PostgreSQL **Platform:** n8n

1. Assessment Overview

This project demonstrates a production-oriented relational database workflow in n8n using PostgreSQL. The workflow is designed to connect to a database, retrieve pending order leads, prevent duplicate processing through transactional row locking, process the selected order, upsert customer information, and update the lead status.

2. Workflow Architecture

Stage	Node	Purpose
1	Manual Trigger	Starts the database workflow.
2	SQL_FetchAndLockRow	Retrieves one pending order lead and applies PostgreSQL row locking.
3	ACTION_ProcessOrder	Processes the selected order data.
4	SQL_UpsertCustomer	Inserts a customer or updates the existing customer by email.
5	SQL_UpdateStatusCompleted	Marks the successfully processed lead as completed.

3. Database Schema Overview

Table	Column	Description
customers	id	Primary key.
customers	email	Unique customer key.
customers	name	Customer name.
order_leads	id	Primary key.
order_leads	customer_email	Customer email; references customers(email).
order_leads	customer_name	Customer name supplied with the order lead.
order_leads	product	Ordered product or plan.
order_leads	amount	Order amount.
order_leads	lead_status	Processing status; seeded as pending.

4. Database Setup & Seed Data

The setup workflow uses a Manual Trigger followed by SQL_CreateSchemaAndSeed. The SQL creates the customers and order_leads tables and inserts sample customers and pending order leads.

```
CREATE TABLE IF NOT EXISTS customers (  
  id SERIAL PRIMARY KEY,  
  email TEXT NOT NULL UNIQUE,  
  name TEXT  
);  
  
CREATE TABLE IF NOT EXISTS order_leads (  
  id SERIAL PRIMARY KEY,  
  customer_email TEXT NOT NULL,  
  customer_name TEXT NOT NULL,  
  product TEXT NOT NULL,  
  amount NUMERIC NOT NULL,  
  lead_status TEXT NOT NULL  
);
```

```
id SERIAL PRIMARY KEY,  
customer_email TEXT NOT NULL REFERENCES customers(email),  
customer_name TEXT,  
product TEXT,  
amount NUMERIC(10,2),  
lead_status TEXT NOT NULL DEFAULT 'pending'  
);  
  
INSERT INTO customers (email, name)  
VALUES  
  ('ada@example.com', 'Ada L.'),  
  ('grace@example.com', 'Grace H.')  
ON CONFLICT (email) DO NOTHING;
```

5. Pending Lead Retrieval & Queue Processing

SQL_FetchAndLockRow selects a single pending record in ID order. PostgreSQL's SKIP LOCKED behavior allows another concurrent worker to skip a row that is already locked instead of processing the same record.

```
SELECT
  id,
  customer_email,
  customer_name,
  product,
  amount
FROM order_leads
WHERE lead_status = 'pending'
ORDER BY id
FOR UPDATE SKIP LOCKED
LIMIT 1;
```

6. Customer Upsert Logic

SQL_UpsertCustomer uses email as the unique customer key. If the email does not exist, a new customer is inserted. If the email already exists, the customer name is updated. This prevents duplicate customer records.

```
INSERT INTO customers (email, name)
VALUES ($1, $2)
ON CONFLICT (email)
DO UPDATE SET name = EXCLUDED.name
RETURNING id, email, name;
```

7. Status Management

After successful order processing and customer upsert, SQL_UpdateStatusCompleted changes the selected lead's lead_status from pending to completed. The update should target the exact lead ID returned by the locking step.

```
UPDATE order_leads
SET lead_status = 'completed'
WHERE id = $1
RETURNING id, lead_status;
```

8. Production Naming Standards

- **SQL_CreateSchemaAndSeed** — one-time database schema and test-data setup.
- **SQL_FetchAndLockRow** — retrieves and locks a pending lead.
- **ACTION_ProcessOrder** — processes the selected order.
- **SQL_UpsertCustomer** — performs email-based customer upsert.
- **SQL_UpdateStatusCompleted** — marks the processed lead completed.

9. Testing & Evidence

- Database connection successfully configured in n8n using PostgreSQL.
- Schema setup workflow executed successfully.
- Sample pending order leads were inserted for testing.
- Pending lead retrieval was implemented with PostgreSQL row-locking syntax.
- Customer upsert behavior was configured using the unique email key.
- Successful processing is followed by a completed status update.
- Execution screenshots should be captured for the LMS submission.

10. Assumptions & Limitations

PostgreSQL was selected because the assessment requires production-oriented transactional locking and upsert behavior. The exact transaction boundary should be verified in the final n8n execution configuration so the row lock remains valid through the intended processing/status-update sequence. Execution screenshots and the final exported workflow JSON are submission evidence and should be attached separately.

11. Final Submission Checklist

- Loom/YouTube demonstration video
- Exported n8n workflow JSON
- This PDF documentation package
- Workflow execution screenshots/evidence
- Database schema / SQL configuration evidence
- Public GitHub repository, if used
- Description/comments covering challenges, assumptions, limitations and improvements

Note: This document is prepared from the assessment requirements and the provided PostgreSQL setup workflow. The final submission should include actual execution evidence from the completed workflow.